The next-to-shortest path problem on directed graphs with positive edge weights

Bang Ye Wu and Hung-Lung Wang*
National Chung Cheng University, ChiaYi, Taiwan 621, R.O.C.
National Taipei College of Business, Taipei, Taiwan 100, R.O.C.

Abstract

For two vertices s and t in a graph $G = (V, E)$, the next-to-shortest path is an st-path whose length is minimum amongst all st-paths strictly longer than the shortest one. In this paper we consider the version that the input graph G is directed. First we show some properties for the problem on general directed graphs with positive edge weights. The second contribution of this paper is an $O(n^3)$-time algorithm for the case where G is a planar directed graph with positive weights, and n is the numbers of vertices of G.

Keywords. Algorithm, Graph, Shortest path, Time complexity, Next-to-shortest path.

1 Introduction

Let $G = (V, E, w)$ be a graph with vertex set V and edge set E. Each edge $(u, v) \in E$ is associated with an edge-length $w(u, v)$. For $s, t \in V$, a *simple st-path* is a path from s to t without repeated vertices in the path. The *length* of a path is the total weights of all edges in the path. An st-path is a shortest st-path if its length is minimum amongst all possible st-paths. The length of the shortest st-path is denoted by $d(s, t)$. A *next-to-shortest* st-path is an st-path whose length is minimum amongst the st-paths of lengths *strictly longer* than $d(s, t)$. For a given instance (G, s, t), the *next-to-shortest path problem* is to find a next-to-shortest st-path.

In [10], the next-to-shortest path problem was first studied by Lalgudi and Papaefthymiou for digraphs (directed graphs) with no restriction to positive edge weights. They showed that the problem is NP-hard but can be efficiently solved if the definition of paths is replaced by *walks*, i.e., allowing repeated vertices. Algorithms for the problem on special graphs were also studied [1, 12]. For undirected graphs $G = (V, E)$ with positive edge weights, the first polynomial-time algorithm was presented in [9] with time complexity $O(n^3m)$ time, in which n and m denote $|V|$ and $|E|$, respectively. The time complexity has been improved several times [8, 11, 18], and the currently best result is a linear time algorithm, assuming that the distances from s and t to all other vertices are given [18]. Hence, for undirected graphs, the positive-weight version of the next-to-shortest path problem can be solved with the same time complexity as the single source shortest paths problem. The problem becomes much more complicated for two generalizations: either zero-weight edges are allowed, or the input graph is directed. In [21], Zhang and Nagamochi showed that the version for undirected graphs with nonnegative weights admits a polynomial-time algorithm with time complexity $O(n^9m)$. Independently, Wu et al. showed in a manuscript [19] that this version can be solved in linear time if the distances from s and t to all other vertices are given. On the other hand, no polynomial-time is known for the directed version unless the input graph is acyclic. In such a case, the algorithm in [10] can be used to solve the problem in linear time. Suppose that there are k shortest st-paths. The next-to-shortest path is in fact the $(k+1)$-th shortest path. After constructing the union of all shortest paths, the number k can be easily computed. Eppstein [4] showed an efficient algorithm for finding the k shortest st-paths in a digraph. However, in his paper, cycles are allowed in a path, that is, the algorithm is in fact for k shortest walks. Another difficulty to apply Eppstein's algorithm to next-to-shortest path is that k may be up to exponential in terms of n.

In this paper, we focus on the variant where the input is a digraph. The problem becomes more complicated when there are cycles. One of the important properties used in the previous works to solve the problem is that a next-to-shortest path

*Corresponding author: hlwang@webmail.ntcb.edu.tw

contains at most one *outward subpath*, i.e., a subpath not in the union of all the shortest st-paths. However, by an example, we show that this property does not hold for digraphs. Fortunately, we also show that these outward subpaths can only be within a special region, and this property is helpful for designing algorithms. By additionally exploring the properties of the problem on planar graphs, we design an $O(n^3)$-time algorithm for planar digraphs with positive edge weights.

The paper is organized as follows. In Section 2, we give some notation and definitions used in this paper, as well as some basic properties. In Sections 3, we show some properties for the problem on general directed graphs, and the case of planar graphs is discussed in Section 4. Finally concluding remarks are given in Section 5.

2 Preliminaries

Throughout this paper, we shall assume that (G, s, t) is the instance of the problem, in which $G = (V, E, w)$ is the input graph with vertex set V, edge (arc) set E and edge length w. Vertices s and t are in V. The graph G is simple, connected and directed. We shall use $n = |V|$ and $m = |E|$. An edge from u to v is denoted by (u, v).

For a graph H, $V(H)$ and $E(H)$ denote the vertex and edge sets, respectively. A uv-path is a path from u to v. A path is *simple* if it does not visit any vertex more than once. Two paths are *disjoint* if they have no common vertex; and *internally disjoint* if they are disjoint except the endpoints. For vertices x and y on path P, let $P[x, y]$ denote the subpath from x to y and $w(P) = \sum_{e\in E(P)} w(e)$ denote the length of the path. We also use "a path $P[u, v]$" to indicate P is a uv-path. Let $d_H(x, y)$ denote the shortest path length from x to y in graph H, which is also called the distance from x to y. Let $d_H(x, y) = \infty$ if there is no xy-path. The subscript will be omitted for simplicity if the distance is on the input graph G. For convenience, let $d_s(v) = d(s, v)$ and $d_t(v) = d(v, t)$.

The distances can be found by solving the single source shortest paths (SSSP) problem. For general directed and positive weight graphs , the SSSP problem can be solved in $O(m + n \log n)$ time [2, 3, 5].

The next-to-shortest path problem is closely related to the 2-disjoint paths problem (2-VDP). For a given graph and k pairs of vertices, the k-disjoint paths problem is to determine if there exist disjoint paths connecting the given k source vertices to their corresponding sink vertices. The k-VDP problem is much more complicated than the problem of determining if there are k disjoint paths between two vertices, which is just the connectivity problem and can be easily solved by, for example, a maximum-flow algorithms. When the number of pairs is fixed, the k-disjoint paths problem is polynomial-time solvable for undirected graphs [13, 14] but NP-complete for general digraphs [6]. For dag, it was known the problem can be solved in polynomial time for fixed k [6]. Recently, Tholey showed a linear time algorithm for 2-VDP on dags in [15]. For planar graph, there exists a simple linear-time algorithm for 2-VDP [16]. Optimization problems about k-VDP have also been studied, for example, [17, 20].

Let D be the union of all shortest st-paths. When all the edge weights are positive or G is a acyclic (the possible settings in this paper), constructing D can be done in linear time as follows. A vertex v is in $V(D)$ iff $d_s(v) + d_t(v) = d(s, t)$, and, for both u and v in $V(D)$, a directed edge $(u, v) \in E(D)$ iff $d_s(v) = d_s(u) + w(u, v)$. For the case where G is a general directed or undirected graph, if all edge lengths are positive, D is a dag. In a dag, we may use the terms such as parent, child, ancestor and descendant as in a rooted tree. We define a binary relation on $V(D)$: $x \prec y$ iff x is an ancestor of y. In our definition, a vertex is not an ancestor of itself. Also define $x \preceq y$ iff x is an ancestor of y or $x = y$.

Apparently s and t are in $V(D)$ and, for any $x, y \in V(D)$, any xy-path in D is a shortest xy-path in G. An *outward path* is path with both endpoints in $V(D)$ and passing through, possibly none, only vertices in $V - V(D)$. Equivalently, an outward path uses only edges in $E - E(D)$ and both endpoints are in $V(D)$. We list some simple properties in the following without proofs.

Fact 1 : An st-path containing an outward subpath is longer than a shortest st-path.

Fact 2 : In a dag, any path leading to a vertex v contains only ancestors of v. Similarly, any path leading from v contains only descendants of v.

Fact 3 : In a dag, if u is not an ancestor of v, any path starting from u contains no ancestor of v.

Fact 4 : In a dag, any uv-path and vx-path are internally disjoint.

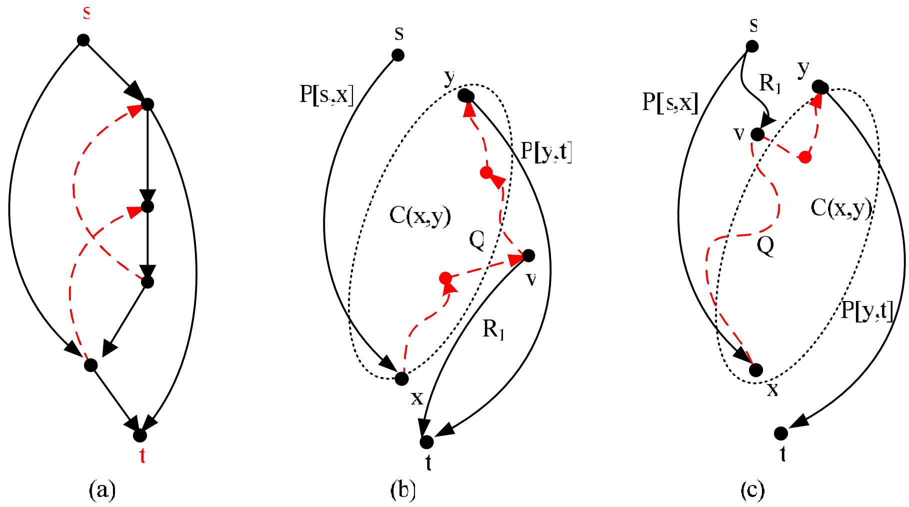

Figure 1: (a): An example of a next-to-shortest path with two outward subpaths. The solid lines are edges in D and the dashed lines are outward paths. The graph has only one next-to-shortest path which contains the two outward subpaths. (b): The case where a mixed subpath contains a vertex which is not an ancestor of x. (c): The case where a mixed subpath contains a vertex which is not a descendant of y.

3 Properties for general digraphs

In this section, we consider the next-to-shortest path on general digraph with positive edge weights. Unlike dags or undirected graphs, a next-to-shortest path on a general digraph may contain more than one outward subpath. In Figure 1.(a), we show such an example. In this section, we shall derive some properties for these outward subpaths.

Definition 1 : A *mixed path* is a path starting and ending with outward subpaths. The *maximal mixed subpath* of a path is the mixed subpath not properly contained in another mixed subpath.

We note that a mixed subpath contains at least one outward subpath, and possibly is just an outward subpath without any other subpath. The next fact is trivial since an *st*-path without any mixed subpath is a shortest path.

Fact 5: A next-to-shortest path contains exactly one maximal mixed subpath.

Definition 2 : For two vertices x and y in D, $C(x,y) = \{v \in D | y \prec v \prec x\} \cup \{x,y\}$. Define $H_{xy} = G[V-(V(D)-C(x,y))]$. The shortest path length from x to y in H_{xy} is denoted by $d^H(x,y)$ for simplicity.

Note that $C(x,y) = \{x,y\}$ if y is not an ancestor of x.

Lemma 6 : If P is a next-to-shortest path with maximal mixed subpath $Q = P[x,y]$, then $V(Q) \cap V(D) \subset C(x,y)$.

Proof: First, we consider that case where there is a vertex in $V(Q) \cap V(D)$, which is not an ancestor of x. Let v be the first such vertex and R_1 any vt-path in D (Figure 1.(b)). We claim that $R = P[s,v] \circ R_1$ is a better solution, which contradicts to the optimality of P. By definition, $P[s,v]$ contains at least one outward subpath and therefore $w(R) > d(s,t)$. Since the vertices in D passed by $P[s,v]$ are all ancestors of x and R_1 contains no ancestor of x, R is simple.

The other case is that all vertices in $V(Q) \cap V(D)$ are ancestors of x. Any vertex in $V(Q) \cap V(D)$ but not in $C(x,y)$ is an ancestor of x but not a descendant of y. Let v be one of those vertices with minimum distance from s and R_1 be any sv-path in D (Figure 1.(c)). We claim that $R = R_1 \circ P[v,t]$ is a better solution. By definition, $P[v,t]$ contains at least one outward subpath and therefore $w(R) > d(s,t)$. Since R_1 passes through only ancestors of v and $P[v,t]$ contains no ancestor of v, R is simple. $\square$

A basic approach for solving the next-to-shortest path problem is to find, for all x and y, a shortest path $P = P[s,x] \circ P[x,y] \circ P[y,t]$, in which $P[s,x]$ and $P[y,t]$ are two disjoint paths in D, and $P[x,y]$ is a mixed path. When edge weights are positive, D is a dag, and the 2-VDP problem can be solve in linear time [15]. By the above lemma, $P[x, y]$ is a path in H_{xy}. But the difficulty is that a solution of the 2-VDP may also contains vertices in H_{xy}, and thus finding an arbitrary shortest xy-path in H_{xy} is not sufficient to solve the problem.

However, we can solve a easier case. The next corollary directly follows from the above lemma and that $C(x, y) = \{x, y\}$ if $y \not\prec x$.

Corollary 7: If P is a next-to-shortest path with maximal mixed subpath $P[x, y]$ such that y is not an ancestor of x, then $P[x, y]$ is an outward path. Furthermore, $P[x, y]$ is a shortest path in H_{xy}.

Note that the above result includes that case that y is a descendant of x.

Lemma 8: Suppose that P is a next-to-shortest path with maximal mixed subpath $P[x, y]$ such that y is not an ancestor of x. If $Q_1[s, x], Q_2[y, t]$ are any two disjoint paths in D and Q_0 is a shortest xy-path in H_{xy}, then $Q = Q_1[s, x] \circ Q_0 \circ Q_2[y, t]$ is a next-to-shortest path.

Proof: By Corollary 7,

$$w(P) = d_s(x) + d_t(y) + d^H(x, y) = w(Q).$$

Since $y \not\prec x$, $Q_1[s, x]$ and $Q_2[y, t]$ are disjoint. Since Q_0 is an outward path, it is internally disjoint to both Q_1 and Q_2. Therefore Q is simple. □

4 On planar digraphs

In this section we consider the next-to-shortest path problem on planar digraphs with positive edge weights. Since a planar digraph is also a digraph, the properties derived in the previous section also hold for planar digraphs. Immediately we have the next result.

Lemma 9: If G is planar and there exists a next-to-shortest path P with maximal mixed subpath $P[x, y]$ such that y is not an ancestor of x, the next-to-shortest path problem can be solved in $O(n^3)$ time.

Proof: By Lemma 8, the length of the next-to-shortest path is

$$\min_{y \not\prec x}(d_s(x) + d_t(y) + d^H(x, y)),$$

and the solution can be constructed in linear time once x, y have been determined. Since G is planar, $d^H(x, y)$ can be computed in linear time. The total time complexity is $O(n^3)$ since there are $O(n^2)$ possible pairs and $O(m) = O(n)$ for planar graphs. □

The complicated case is that $y \prec x$, i.e., the mixed subpath is from a vertex to its ancestor. It is well-known that a planar graph can be embedded in a plane such that no edge will cross another. In the following we shall discuss the problem on a fixed plane-embedding of G. For convenience, we assume that a vertex is upper than all its descendants, if it has, in D. Consequently, s is the upmost and t is the lowest vertices in the embedding.

Definition 3: Suppose that G is a planar graph embedded in a plane and $y \prec x$ are two vertices in D. The region of $C(x, y)$ is the minimal region of the plane containing all the vertices in $C(x, y)$. The *boundary* B of $C(x, y)$ is composed of the vertices and edges which are outmost in the region. The boundary can be divided into the left and the right sub-boundaries, which are two yx-paths in D. For two vertices $u \preceq v$ on the same sub-boundary, let $B[u, v]$ denote the unique subpath along B.

The left and the right sub-boundaries share at least two vertices y and x. It is possible that they have common edges and intersects several times. That is, they may have one or more common subpaths, which are referred to as *overlapping boundaries.* It also happens that the entire boundary is an overlapping boundary.

Definition 4: A vertex on the boundary is an *entrance*, and an *exit* respectively, if it has a parent, and a child respectively, in D outside the region $C(x, y)$. An entrance is a *left/right entrance* if the corresponding edge from its parent is on the left/right side of the region. The left/right exits are defined analogously.

We note that the vertex on an overlapping boundary, such as y and x, may be simultaneously a left and a right entrance/exit. The definitions are depicted in Figure 2.(a).

Definition 5: For an entrance λ and an exit α of $C(x, y)$, the ordered pair (λ, α) is *valid* for (x, y) if there are two disjoint sx-path P_1 and yt-path P_2 in D such that $P_1 = P_1[s, \lambda] \circ B[\lambda, x]$ and $P_2 = B[y, \alpha] \circ P_2[\alpha, t]$.

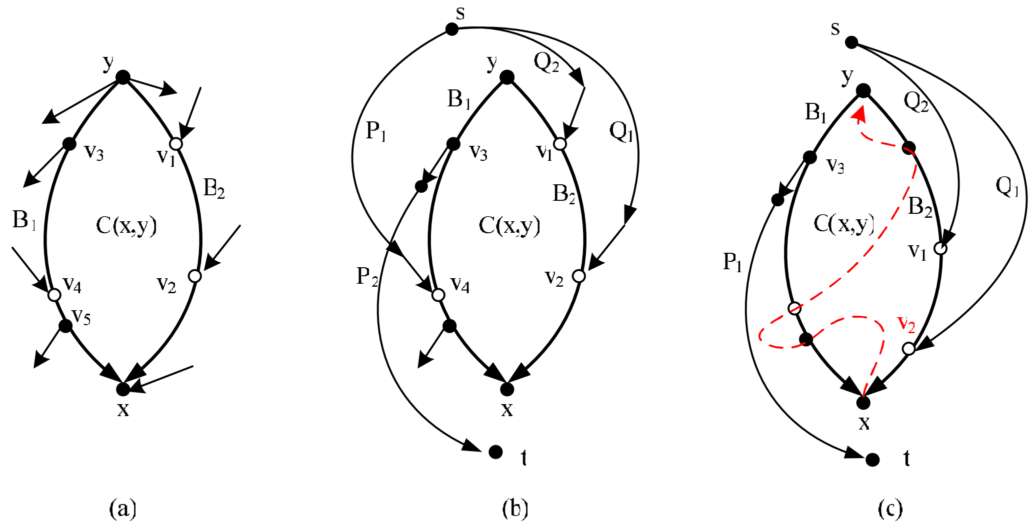

Figure 2: (a) An illustration of the region $C(x, y)$. The boundary is divided into the left sub-boundary B_1 and the right sub-boundary B_2. v_1, v_2 are right entrances and v_4 is a left entrance. Both v_3 and v_5 are left exits. y is simultaneously a left and a right exit, and x is a right entrance but not a left entrance. (b) An illustration of Fact 10. When $B(\lambda, x)$ and $B(y, \alpha)$ are disjoint, an entrance and an exit on the same side cannot be valid, such as (v_4, v_3); and an entrance and an exit on different sides must be valid, such as (v_1, v_3) and (v_2, v_3). (c) An example of a next-to-shortest path with maximal mixed subpath from x to y. The dashed path is the mixed subpath. Since (v_1, v_3) is a valid pair, (v_2, v_3) is also valid if v_2 is the lowest entrance. Note that the mixed subpath cannot intersect $B_2[v_2, x]$ if it does not intersect $B_2[v_1, x]$.

Notice that (x, y) can be valid for (x, y), but (y, x) cannot. The next result directly follows the property of a plane embedding, see Figure 2.(b) for an illustration.

Fact 10 : The pair (λ, α) is valid for (x, y) if and only if

- $B(\lambda, x)$ and $B(y, \alpha)$ are disjoint; and
- λ is a left entrance and α is a right exit; or λ is a right entrance and α is a left exit.

Note that the first condition is necessary. In the case where $\lambda \prec \alpha$ and there is an overlapping boundary between them, (λ, α) is not valid even if they are on different sides.

Fact 11 : Suppose that P is a next-to-shortest path with maximal mixed subpath $P[x, y]$ such that $y \prec x$. Then P passes through an entrance λ and an exit α such that (λ, α) is valid for (x, y). Furthermore $P[x, y]$ does not intersect $B[\lambda, x]$ or $B[y, \alpha]$.

It is possible that $B[\lambda, x]$ is not a subpath of P. Instead P may go from λ to x by using a path Q within the region but not on the boundary. However, since $P[x, y]$ does not intersect Q, $P[x, y]$ also cannot intersect $B[\lambda, x]$ even by outward paths because G is planar. We note that this fact does not hold for non-planar graphs.

Definition 6 : An left exit is *left upmost* if there is no other left exit upper than it. A left entrance is *left lowest* if there is no other left entrance lower than it. The right upmost exit and the right lowest entrance are defined analogously.

Notice that there may be at most two upmost exits and also at most two lowest entrances.

Fact 12 : Suppose that (λ, α) is valid for (x, y). If λ' is the lowest entrance on the same side of λ and α' is the upmost exit on the same side of α, then (λ', α') is also valid.

By the above fact, we can focus on the solution passing through a lowest entrance and an upmost exit. Recall that H_{xy} is the subgraph of G induced by $V - (V(D) - C(x, y))$, i.e., we remove the vertices in D except those in $C(x, y)$. For valid (λ, α) such that λ is lowest and α is upmost, we define $H_{xy}^{\lambda\alpha}$ to be the subgraph of H_{xy} obtained by removing the vertices in $B_1 = B[\lambda, x]$ and $B_2 = B[y, \alpha]$ except x and y, i.e, the subgraph induced by

$$(V - V(D)) \cup (C(x, y) - V(B_1) - V(B_2)) \cup \{x, y\}.$$

Lemma 13: Suppose that P is a next-to-shortest path with maximal mixed subpath $P[x, y]$ such that $y \prec x$ and P passes through a lowest entrance λ and an upmost exit α of $C(x, y)$. Then, $w(P) = d_s(x) + d_t(y) + d_{H'}(x, y)$, in which $H' = H_{xy}^{\lambda\alpha}$. Furthermore, if $Q_1[s, \lambda]$ and $Q_2[\alpha, t]$ are any two disjoint paths in $(D - C(x, y)) \cup \{\lambda, \alpha\}$ and Q_0 is a shortest xy-path in H', then $Q = Q_1 \circ B[\lambda, x] \circ Q_0 \circ B[y, \alpha] \circ Q_2$ is a next-to-shortest path.

Note that a next-to-shortest path may pass through vertices in $V - V(D)$ which are not in the region $C(x, y)$ (Figure 2.(c)).

Theorem 14: When G is a planar digraph with positive edge weights, the next-to-shortest path problem can be solved in $O(n^3)$ time.

Proof: By Lemma 8, the case of $y \not\prec x$ can be solved in $O(n^3)$ time. By Lemma 13, for all $y \prec x$, it is sufficient to compute $d_{H'}(x, y)$, in which $H' = H_{xy}^{\lambda\alpha}$ and (λ, α) is a valid pair of a lowest entrance and an upmost exit. For each pair (x, y), there are at most two such pairs. By Fact 10, checking the validity of a pair can be done in linear time. Since computing the shortest path length can be done in $O(n)$ time in a planar graph, the total complexity is also $O(n^3)$. $\square$

5 Concluding remarks

In this paper, we show that the next-to-shortest path problem on planar digraphs with positive edge weights can be solved in polynomial time. An important open problem is the complexity for the problem on general digraphs with positive weights. Although the properties in Section 3 hold, we did not find an algorithm for the case where $y \prec x$. For planar graphs, we can show that the mixed subpath cannot intersect the sub-boundaries as in Fact 11. But this does not hold for general digraphs. Although the 2-VDP problem on D can be solved in linear time [15] since D is a dag, the difficulty is that there are possibly many two-paths (feasible solutions of 2-VDP), only one of which is used by the optimal solution. If we construct another two-paths, the length of the mixed subpath may be different, and thus not optimal.

Acknowledgment

B.Y. Wu was supported in part by NSC 100-2221-E-194-036-MY3 and NSC 101-2221-E-194-025-MY3, and Hung-Lung Wang was supported in part by NSC 101-2221-E-141-002-, from the National Science Council, Taiwan, R.O.C.

References

[1] Barman, S.C., Mondal S., Pal, M.: An efficient algorithm to find next-to-shortest path on trapezoid graphs. Adv. Appl. Math. Anal. 2, 97–107 (2007)

[2] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. MIT Press and McGraw-Hill (2001)

[3] Dijkstra, E.W.: A note on two problems in connection with graphs. Numer. Math. 1, 269–271 (1959)

[4] Eppstein, D.: Finding the k shortest paths. SIAM J. Comput. 28, 625–673 (1998)

[5] Fredman, M.L., Tarjan, R.E.: Fibonacci heaps and their uses in improved network optimization algorithms. J. ACM 34, 209–221 (1987)

[6] Fortune, S., Hopcroft, J., Wyllie, J.: The directed subgraph homeomorphism problem, Theor. Comput. Sci. 10, 111–121 (1980)

[7] Henzinger, M.R., Klein, P., Rao, S., Subramanian, S.: Faster shortest-path algorithms for planar graphs. J. Comput. Syst. Sci. 53, 2–23 (1997)

[8] Kao, K.-H., Chang, J.-M., Wang, Y.-L., Juan, J.S.-T.: A quadratic algorithm for finding next-to-shortest paths in graphs. Algorithmica 61, 402–418 (2011)

[9] Krasiko, I., Noble, S.D.: Finding next-to-shortest paths in a graph. Inform. Process. Lett. 92, 117–119 (2004)

[10] Lalgudi, K.N., Papaefthymiou, M.C.: Computing strictly-second shortest paths. Inform. Process. Lett. 63, 177–181 (1997)

[11] Li, S., Sun, G., Chen, G.: Improved algorithm for finding next-to-shortest paths. Inform. Process. Lett. 99, 192–194 (2006)

[12] Mondal, S., Pal, M.: A sequential algorithm to solve next-to-shortest path problem on circular-arc graphs. J. Phys. Sci. 10, 201–217 (2006)

[13] Robertson, N., Seymour, P.D.: Graph minors. XIII. The disjoint paths problem, J. Comb. Theory B 63, 65–110 (1995)

[14] Tholey, T.: Solving the 2-disjoint paths problem in nearly linear time, Theor. Comput. Syst. 39, 51–78 (2006)

[15] Tholey, T.: Linear time algorithms for two disjoint paths problems on directed acyclic graphs. Theor. Comput. Sci. 465, 35–48 (2012)

[16] Woeginger, G.: A simple solution to the two paths problem in planar graphs. Inform. Process. Lett. 36, 191–192 (1990)

[17] Wu, B. Y.: A note on approximating the min-max vertex disjoint paths on directed acyclic graphs. J. Comput. Syst. Sci. 77, 1054–1057 (2011)

[18] Wu, B. Y.: A simpler and more efficient algorithm for the next-to-shortest path problem. Algorithmica 65, 467–479 (2013)

[19] Wu, B. Y., Guo, J.-L., Wang, Y.-L.: A linear time algorithm for the next-to-shortest path problem on undirected graphs with nonnegative edge lengths. manuscript available at http://arxiv.org/abs/1203.5235, Submitted to arXiv on 23 Mar 2012

[20] Yu, C.-C., Lin, C.-H., Wang, B.-F.: Improved algorithms for finding length-bounded two vertex-disjoint paths in a planar graph and minmax k vertex-disjoint paths in a directed acyclic graph. J. Comput. Syst. Sci. 76(8), 697–708 (2010)

[21] Zhang, C., Nagamochi, H.: The next-to-shortest path in undirected graphs with nonnegative weights. Proceedings of the Eighteenth Computing: The Australasian Theory Symposium (CATS 2012), Melbourne, Australia, 13–19 (2012)